

Atividade 8

Pipeline Streaming com PySpark e Kafka

Ingestão • Streaming • Enriquecimento SQL • Persistência



Grupo 6

Engenharia de Dados + Big Data • Escola Politécnica da USP

https://github.com/EdCruzJunior/eEDB-022-2026-3-Ingestao_Pipeline_Dados/tree/main/Atividade_8_Ingestao_ETL_Pipeline_Streaming_PySpark_Kafka

1. Objetivo da atividade

Requisito central

Construir um pipeline de streaming local em Docker no qual o produtor lê arquivos do disco, o Kafka atua como camada de mensageria e o consumidor PySpark processa micro-batches.

- Produtor: script Python responsável pela leitura dos arquivos anexados.
- Mensageria: Apache Kafka disponibilizado localmente via Docker.
- Consumidor: job PySpark usando Structured Streaming.
- Enriquecimento: consulta ao PostgreSQL durante o processamento do micro-batch.
- Saída: dados tratados e enriquecidos gravados no disco local em Parquet.

2. Arquitetura implementada



Ponto técnico importante

O Kafka não processa a regra de negócio. Ele transporta e desacopla os eventos. O PySpark é responsável pelo processamento e o PostgreSQL fornece dados de referência para o enriquecimento.

3. Organização do projeto e dados

```
Atividade_8_Ingestao_ETL_Pipeline_Streaming_PySpark_Kafka/  
├── data/  
├── producer/  
│   ├── producer.py  
│   └── requirements.txt  
├── consumer/  
│   └── consumer.py  
├── db/  
│   └── init_db.py  
├── output/  
├── checkpoint/  
├── docker-compose.yml  
└── README.md
```

- Dados trimestrais de 2021 e 2022 utilizados como eventos de entrada.
- EnquadramentoInicia_v2: dimensão para complementar as instituições.
- Glassdoor consolidado: dimensão complementar de indicadores.
- A pasta output recebe o resultado final em Parquet.
- A pasta checkpoint mantém o estado do Structured Streaming.

4. Subida da infraestrutura Docker

```
docker compose up -d kafka postgres  
docker compose run --rm kafka-init  
docker compose run --rm db-init
```

Kafka

Broker local + tópico `reclamacoes` com 3 partições.

PostgreSQL

Banco local usado como fonte de dados de referência para o enriquecimento.

Docker Compose

Orquestra os serviços e cria uma rede interna na qual o Spark pode acessar `kafka:9092` e `postgres:5432`.

5. Produtor Python → Kafka

```
producer.send(  
    "reclamacoes",  
    value=event  
)  
  
producer.flush()
```

- Percorre os CSVs disponíveis em /data.
- Cada linha é transformada em um evento JSON.
- Inclui metadados: event_id, arquivo, linha e timestamp.
- Publica cada evento no tópico `reclamacoes`.
- Ao final, realiza flush para garantir o envio.

Execução observada

O produtor finalizou com **952 mensagens enviadas**.

6. Evidência da execução do produtor

```

powershell - Atividade_8_Ingestao_ETL_Pipeline_Streaming_PySpark_Kafka - Visual Studio Code
Update + - | 🔍 📄 ... - 🗑️ ✕

PS C:\Users\edvaldo.cruz\Documents\Engenharia_de_Dados\eEDB-022-2026-3-Ingestao_Pipeline_Dados\Atividade_8_Ingestao_ETL_Pipeline_Streaming_PySpark_Kafka> docker compose r,
un --rm producer
  Downloading pytz-2026.3.post1-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas==2.2.3->-r /app/requirements.txt (line 2))
  Downloading tzdata-2026.4-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas==2.2.3->-r /app/requirements.txt (line 2))
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
  Downloading kafka_python-2.0.2-py2.py3-none-any.whl (246 kB)
    _____ 246.5/246.5 kB 24.1 MB/s eta 0:00:00
  Downloading pandas-2.2.3-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (13.1 MB)
    _____ 13.1/13.1 MB 18.2 MB/s eta 0:00:00
  Downloading numpy-2.4.6-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.9 MB)
    _____ 16.9/16.9 MB 7.7 MB/s eta 0:00:00
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
    _____ 229.9/229.9 kB 15.1 MB/s eta 0:00:00
  Downloading pytz-2026.3.post1-py2.py3-none-any.whl (508 kB)
    _____ 508.3/508.3 kB 28.6 MB/s eta 0:00:00
  Downloading tzdata-2026.4-py2.py3-none-any.whl (347 kB)
    _____ 347.5/347.5 kB 23.6 MB/s eta 0:00:00
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: pytz, kafka-python, tzdata, six, numpy, python-dateutil, pandas
Successfully installed kafka-python-2.0.2 numpy-2.4.6 pandas-2.2.3 python-dateutil-2.9.0.post0 pytz-2026.3.post1 six-1.17.0 tzdata-2026.4
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual env
ironment instead: https://pip.pypa.io/warnings/venv

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
[PRODUTOR] Kafka=kafka:9092 tópico=reclamacoes
[PRODUTOR] Arquivos: ['2021_tri_01(7).csv', '2021_tri_02(7).csv', '2021_tri_03(7).csv', '2021_tri_04(7).csv', '2022_tri_01(7).csv', '2022_tri_03(7).csv', '2022_tri_04(7).
csv', 'glassdoor_consolidado_join_match_v2(6).csv']
[PRODUTOR] enviados=100
[PRODUTOR] enviados=200
[PRODUTOR] enviados=300
[PRODUTOR] enviados=400
[PRODUTOR] enviados=500
[PRODUTOR] enviados=600
[PRODUTOR] enviados=700
[PRODUTOR] enviados=800
[PRODUTOR] enviados=900
[PRODUTOR] Finalizado. Total de mensagens enviadas: 952
PS C:\Users\edvaldo.cruz\Documents\Engenharia_de_Dados\eEDB-022-2026-3-Ingestao_Pipeline_Dados\Atividade_8_Ingestao_ETL_Pipeline_Streaming_PySpark_Kafka>

```

7. Consumidor PySpark Structured Streaming

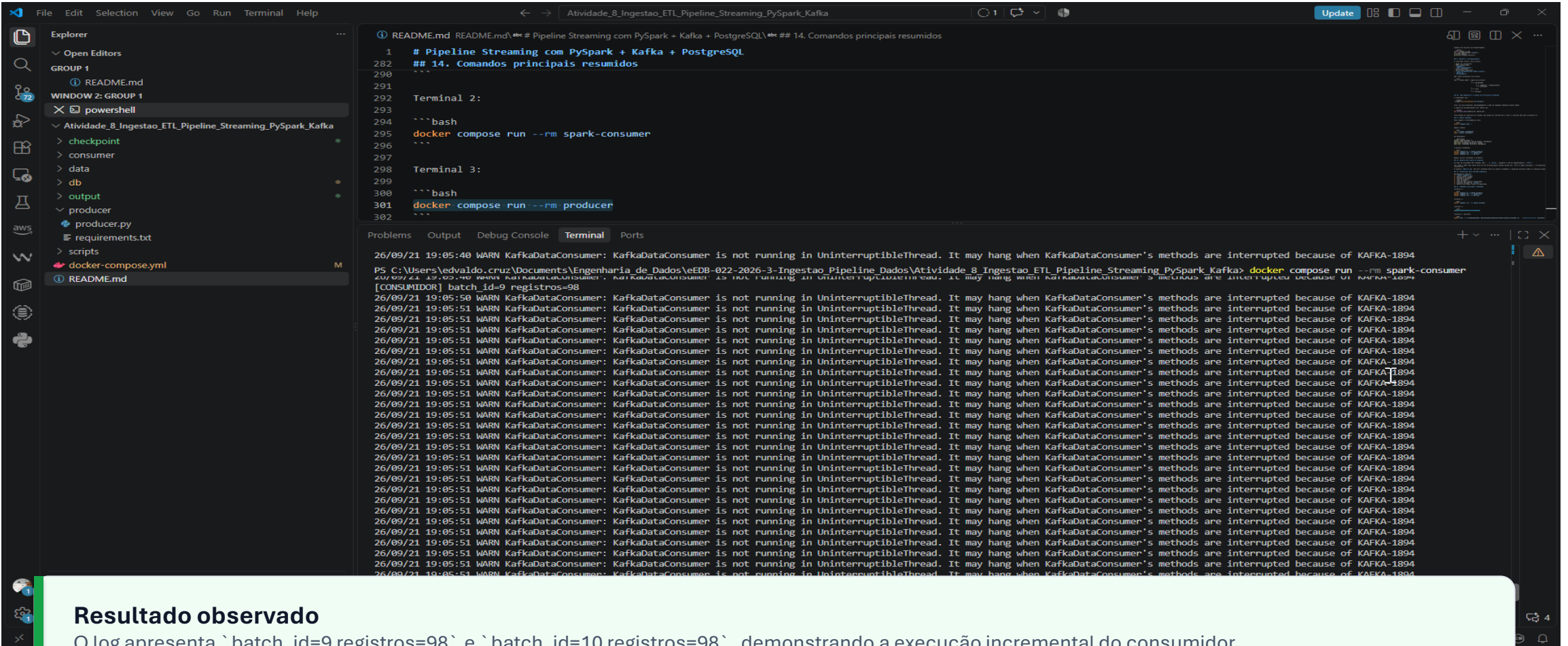
```
raw = (  
    spark.readStream  
        .format("kafka")  
        .option("kafka.bootstrap.servers", "kafka:9092")  
        .option("subscribe", "reclamacoes")  
        .option("startingOffsets", "earliest")  
        .load()  
)
```

- Spark lê continuamente o tópico Kafka.
- A mensagem JSON é desserializada conforme um schema.
- O processamento usa `foreachBatch`.
- A janela é controlada por micro-batches de aproximadamente 10 segundos.
- O checkpoint registra o estado do streaming.

Modelo de execução

Não é necessário esperar todo o arquivo terminar para processar. O consumidor trabalha continuamente com os eventos que chegam ao Kafka.

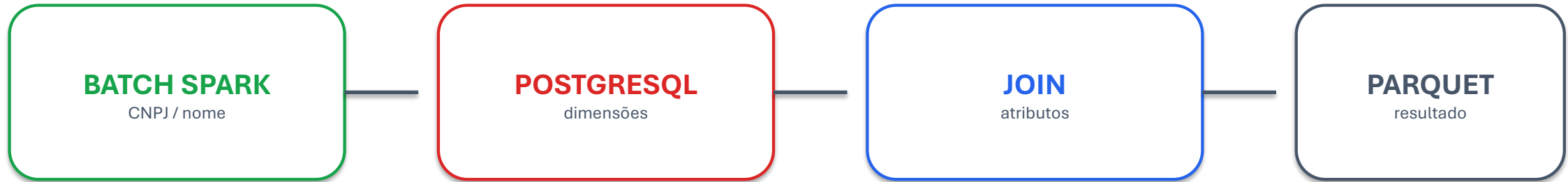
8. Processamento por micro-batch



The screenshot shows a VS Code editor with a project named 'Atividade_8_Ingestao_ETL_Pipeline_Streaming_PySpark_Kafka'. The file explorer on the left shows a 'docker-compose.yml' file. The main editor displays a README.md file with instructions for running the pipeline. The terminal at the bottom shows the output of the command `docker compose run --rm spark-consumer`. The logs indicate that the KafkaDataConsumer is not running in UninterruptibleThread, which may cause it to hang when its methods are interrupted. The logs also show a successful batch processing of 98 records for batch_id=9.

Resultado observado
O log apresenta `batch\_id=9 registros=98` e `batch\_id=10 registros=98`, demonstrando a execução incremental do consumidor.

9. Enriquecimento com PostgreSQL



- CNPJ é normalizado para facilitar a correspondência.
- Tabela `institution\_enrichment` : segmento e nome da instituição.
- Tabela `glassdoor\_enrichment` : indicadores complementares.
- O resultado é associado ao micro-batch recebido pelo Spark.
- Também é calculada a taxa de reclamações por 100 mil clientes.

10. Persistência do resultado

```
enriched.write  
    .mode("append")  
    .parquet(OUTPUT)
```

- Formato: Parquet.
- Diretório: `output/`.
- Modo: append, preservando os micro-batches processados.
- Resultado pronto para consulta analítica posterior.
- A persistência separa o processamento do consumo analítico.

Campos enriquecidos

segmento • nome_enquadramento • indicadores Glassdoor • taxa_reclamacoes_por_100k_clientes • batch_id • processed_ts



12. Conceitos acadêmicos demonstrados

Desacoplamento

Kafka permite que produtor e consumidor evoluam e operem de forma independente.

Streaming

Structured Streaming processa eventos continuamente por micro-batches.

Mensageria

O tópico funciona como canal persistente de eventos entre sistemas.

Enriquecimento

Dados de eventos são combinados com dados de referência via SQL.

Idempotência / estado

Checkpoint e identificadores de evento ajudam a controlar o processamento.

Formato analítico

Parquet organiza a saída para consumo posterior e análise.

13. Validação e evidências para entrega

- ☒ Infraestrutura Docker ativa
- ☒ Tópico Kafka `reclamacoes` criado
- ☒ 952 mensagens publicadas pelo produtor
- ☒ PySpark conectado ao Kafka
- ☒ Micro-batches processados
- ☒ PostgreSQL carregado com dimensões
- ☒ Dados enriquecidos gravados em Parquet
- ☒ Capturas de tela preservadas como evidência

Evidência quantitativa

Produtor: 952 mensagens

Consumidor: micro-batches observados no log

Saída: arquivos Parquet no diretório output/

14. Kafka UI — Monitoramento do Streaming

Visualizando tópicos, mensagens, consumers e offsets no seu ambiente local



Pipeline Streaming
PySpark + Kafka + PostgreSQL

UI for Apache Kafka 83b5a60 v0.7.2

Cluster: local-kafka

Topics / **reclamacoes**

Overview Messages Consumers Settings Statistics

Seek Type: Offset Offset Partitions: All items are selected. Key Serde: String Value Serde: String

Clear all Submit Oldest First

Search + Add Filters

DONE 116 ms 589 KB 824 messages consumed

Offset	Partition	Timestamp	Key	Preview	Value
0	2	9/22/2026, 09:57:01			{"event_id": "c86c02cc-31f3-4a15-88b3-9f7d70a..."}
0	0	9/22/2026, 09:57:01			{"event_id": "98c7c2ec-2f30-4eb9-94be-c5a177..."}
1	2	9/22/2026, 09:57:02			{"event_id": "6ed7c94b-f603-4450-8918-df6b59..."}
2	2	9/22/2026, 09:57:02			{"event_id": "20a7acfe-4c41-478c-908c-e14e70..."}

1 Topic

Canal lógico onde os eventos são publicados. No exemplo: *reclamacoes* (criado pelo script de inicialização).

2 Partition

O tópico é dividido em partições para permitir paralelismo. No exemplo: 3 partições (0, 1 e 2).

3 Offset

Identificador sequencial de cada mensagem dentro de uma partição. Indica a posição da mensagem e é usado pelos *consumers* para controlar o progresso da leitura.

4 Mensagem (Value)

Conteúdo realmente enviado pelo *producer*. No exemplo, um JSON com os dados da reclamação (*event_id*, etc.).

5 Consumer(s)

Aplicação(ões) que leem as mensagens do tópico. Podem ser vistas na aba "Consumers". Exemplo: o job PySpark (Structured Streaming).

Como acessar

- ✓ Acesse: <http://localhost:8080>
- ✓ Selecione o cluster: local-kafka
- ✓ Vá em Topics → *reclamacoes*
- ✓ Utilize a aba Messages para visualizar as mensagens
- ✓ Use a aba Consumers para ver os consumers conectados

Informações do tópico (Overview)

- ✓ Partitions: 3
- ✓ Replication Factor: 1
- ✓ URP: 0
- ✓ In Sync Replicas: 3 of 3
- ✓ Message Count: (total de mensagens, ex.: 952 ou 1904)

Benefícios

- ✓ Visibilidade em tempo real do streaming
- ✓ Facilita testes e validação da ingestão
- ✓ Permite inspecionar mensagens e offsets
- ✓ Ajuda na análise e explicação acadêmica
- ✓ Interface web intuitiva e fácil de usar

O que foi entregue

Um pipeline de ingestão e processamento streaming totalmente local, reproduzível em Docker, com Kafka como mensageria, PySpark como motor de processamento, PostgreSQL para enriquecimento e Parquet como saída.

- O produtor converteu os registros dos arquivos em eventos Kafka.
- O consumidor recebeu os eventos em micro-batches.
- O PostgreSQL forneceu informações complementares.
- O Spark gerou dados tratados e enriquecidos.
- O resultado foi persistido localmente em formato analítico.